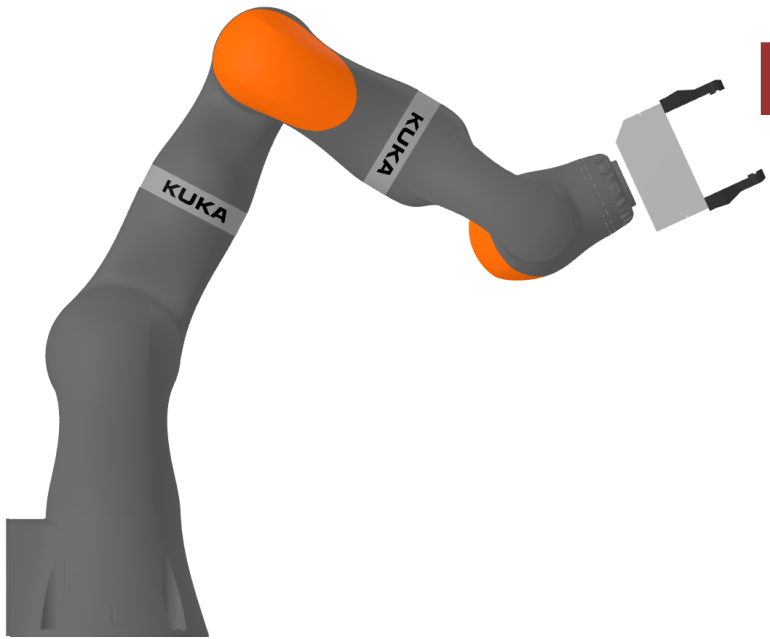


Motion Planning

Part III: Global optimization

MIT 6.4210/2

Robotic Manipulation

Fall 2025, Lecture 11

Massachusetts Institute of Technology

Reminders

- Oct 13 - No lecture
- Oct 16 or Nov 13 - 6.4212 Pset 9
- Oct 17 - Project pre-proposal
- Oct 31 - Project proposal
- Nov 5 - Midterm (7-9 pm) [No lecture]

Review

- Configuration space vs Workspace
- Complete, Prob. Complete, Resolution Complete
- Optimal, Asymptotically Optimal, Approximate
- Roadmap methods: visibility graph, PRM, Canny, GCS
- Tree methods: RRT, RRT-Connect, Grid

Asymptotically Optimal Sampling Based Planners

RRT*

Disadvantage of RRT:

- Not optimal path
- Often long and unnatural zig-zag path

Idea:

- Alter the connectivity of the tree during construction
- Smoothen the tree branches in each time steps such that

If number of nodes reaches infinity: The returned path optimizes some cost function

RRT*

- Asymptotically optimal
- Main idea:
  - Swap new point in as parent for nearby vertices who can be reached along shorter path through new point than through their original (current) parent

RRT*

Algorithm

T.init(q_I)

For k=1 to N:

$q_r \leftarrow$ random_sample_C_{free}()

$q_n \leftarrow$ nearest_neighbor(q_r, T)

if new_state($q_r, q_n, q_c, u, \Delta t$)

T.add_vertex(q_c)

T.add_edge(q_n, q_c, u)

Return T

$q_r \leftarrow$ random_sample_C_{free}()

$q_n \leftarrow$ nearest_neighbor(q_r, T)

if new_state($q_r, q_n, q_c, u, \Delta t$)

$N \leftarrow$ neighbors(T, q_c)

$q_p \leftarrow$ choose_parent(N, q_n, q_c, u)

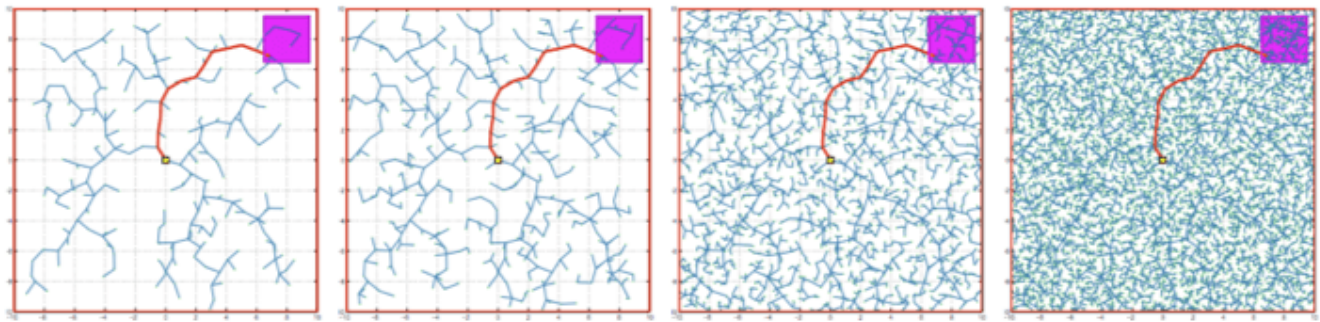
T.add_vertex(q_c)

T.add_edge(q_p, q_c, u)

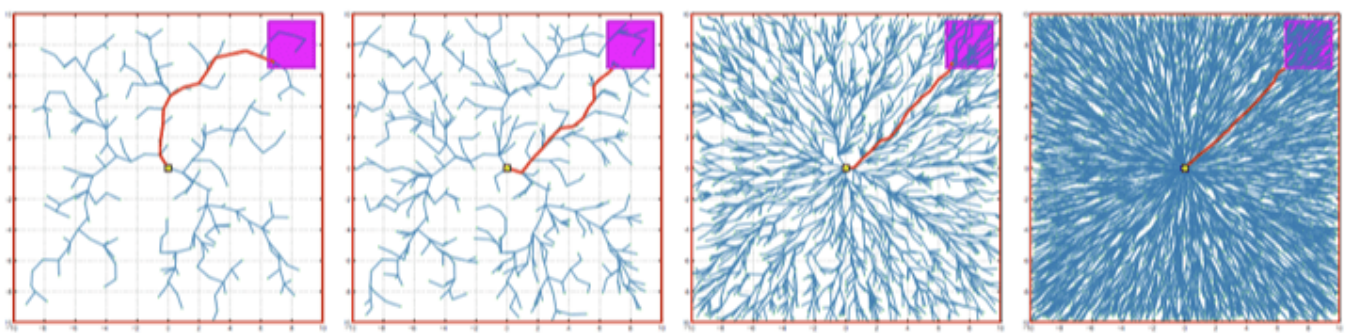
T.rewire(T, N, q_p, q_c)

RRT*

RRT



RRT*



Source: Karaman and Frazzoli

AORRTC: Almost-Surely Asymptotically Optimal Planning with RRT-Connect

Tyler S. Wilson[1], Wil Thomason[2], Zachary Kingston[3], and Jonathan D. Gammell[1]

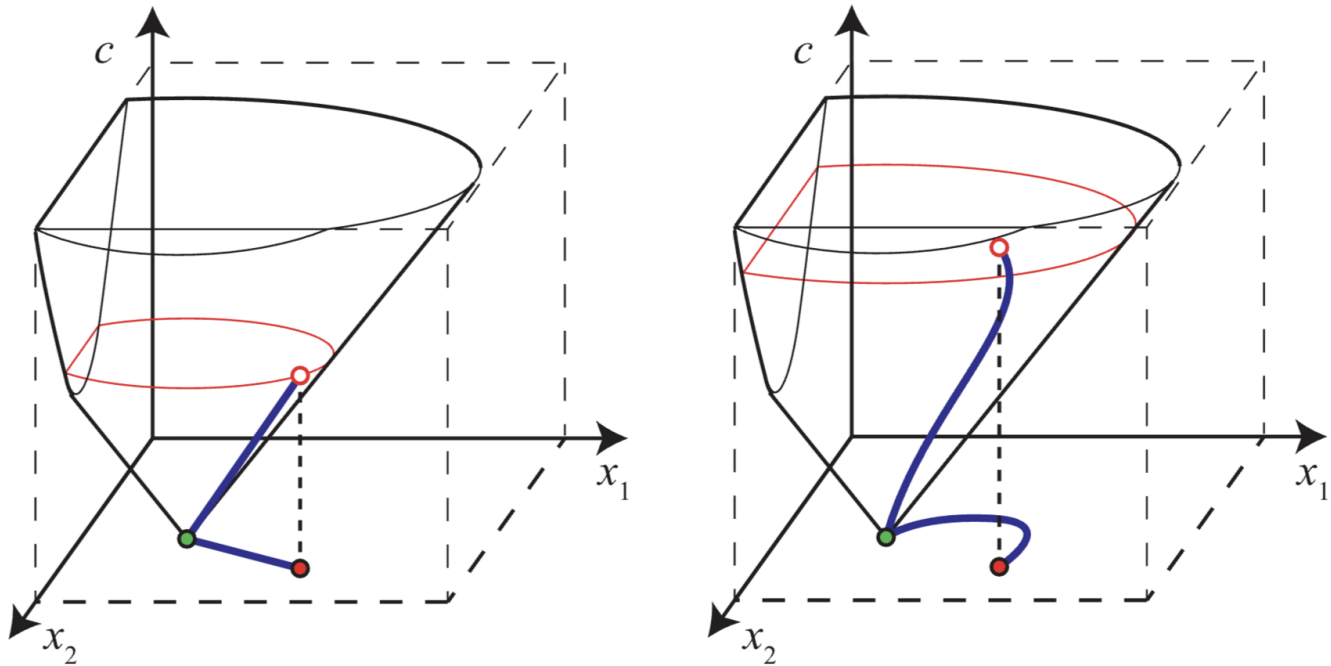


Fig. 1. State–cost space for a 2-D kinematically constrained problem with a path length cost function. State–cost space is 3-D, with a conical reachable set with an apex at the start configuration (green circle). Two paths to the same state–space target (red filled circles) follow trajectories that arrive at different points in state–cost space (red open circles).

Hauser & Zhou 2016 AO-x

https://www.youtube.com/embed/j1itxP3KuiM?enablejsapi=1

Algorithm 1: AORRTC

Function `aorrtc()`
- $\sigma_0 \leftarrow$ `simplify(rrt-connect(`∞`))`;
- $\sigma_{\text{best}} \leftarrow \sigma_0$; $c_{\min} \leftarrow c(\sigma_0)$;
- **repeat**
  - $\sigma_i \leftarrow$ `simplify(rrt-connect(`$c_{\min}$`))`;
  - **if** $\sigma_i \not\equiv \emptyset$ **then**
    - $\sigma_{\text{best}} \leftarrow \sigma_i$;
    - $c_{\min} \leftarrow c(\sigma_i)$;
- **until** `stop`;
- **return** σ_{best};

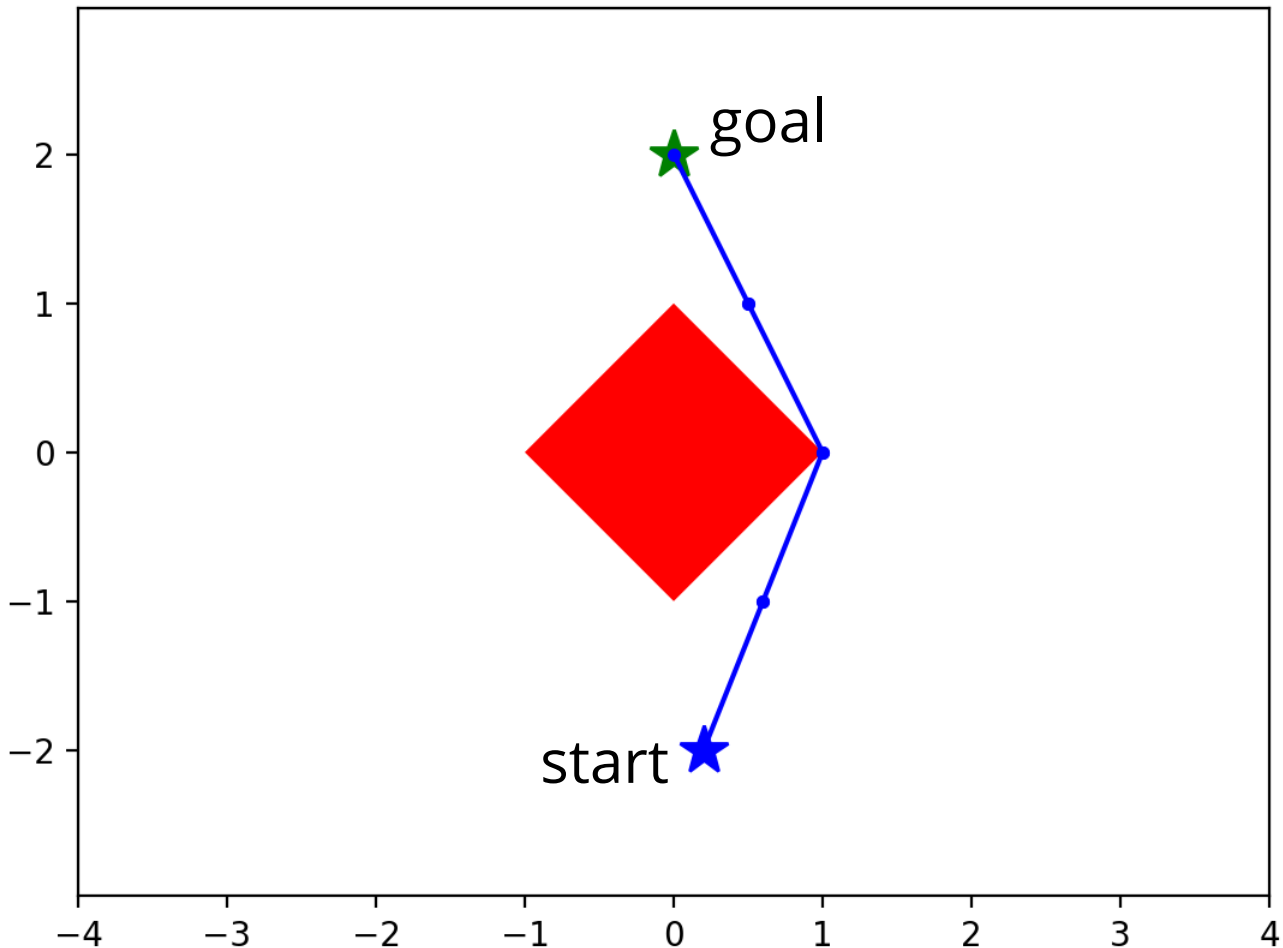
Function `rrt-connect(`$c_{\max}$`)`
- $V_a \leftarrow \{(x_{\text{start}}, 0)\}$; $E_a \leftarrow \emptyset$; $T_a := \{V_a, E_a\}$;
- $V_b \leftarrow \{(x_{\text{goal}}, 0)\}$; $E_b \leftarrow \emptyset$; $T_b := \{V_b, E_b\}$;
- **repeat**
  - $x_{\text{rand}} \sim \mathcal{U}\left(X_{\hat{f}}\right)$;
  - $c_{\text{rand}} \sim \mathcal{U}\left(\left(\hat{g}_{T_a}(x_{\text{rand}}), c_{\max} - \hat{h}_{T_a}(x_{\text{rand}})\right)\right)$;
  - $x_{\text{near}} \leftarrow$ `nearest(`$T_a, x_{\text{rand}}, c_{\text{rand}}$`)`;
  - $x_{\text{new}} \leftarrow$ `steer(`$x_{\text{near}}, x_{\text{rand}}$`)`;
  - **if** `validate(`$x_{\text{near}}, x_{\text{new}}$`)` **then**
    - $c_{\text{new}} \leftarrow$ `extend(`$T_a, x_{\text{near}}, x_{\text{new}}$`)`;
    - **if** `connect(`$T_b, x_{\text{new}}, c_{\text{new}}$`)` **then**
      - **return** `path(`T_a, T_b`)`;
  - `swap(`T_a, T_b`)`;
- **until** `timeout`;
- **return** $\emptyset$;

Function `extend(`$T, x_{\text{near}}, x_{\text{new}}$`)`
- **repeat**
  - $x_{\text{p}} \leftarrow x_{\text{near}}$;
  - $c_{\text{new}} \leftarrow g_T(x_{\text{p}}) + \hat{c}(x_{\text{p}}, x_{\text{new}})$;
  - $c_{\text{rand}} \sim \mathcal{U}\left((\hat{g}_T(x_{\text{new}}), c_{\text{new}})\right)$;
  - $x_{\text{near}} \leftarrow$ `nearest(`$T, x_{\text{new}}, c_{\text{rand}}$`)`;
- **until** $x_{\text{near}} \equiv x_p$ **or not** `validate(`$x_{\text{near}}, x_{\text{new}}$`)`;
- $E \xleftarrow{+} (x_{\text{p}}, x_{\text{new}})$;
- $V \xleftarrow{+} (x_{\text{new}}, c_{\text{new}})$;
- **return** c_{new};

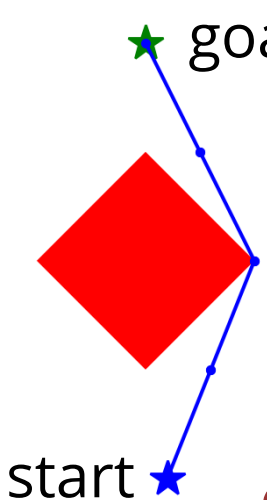
Function `connect(`T_b, x, c`)`
- **repeat**
  - $x_{\text{near}} \leftarrow$ `nearest(`$T_b, x, c_{\max} - c$`)`;
  - $x_{\text{new}} \leftarrow$ `steer(`x_{near}, x`)`;
  - **if** `validate(`$x_{\text{near}}, x_{\text{new}}$`)` **then**
    - `extend(`$T_b, x_{\text{near}}, x_{\text{new}}$`)`;
- **until** $x_{\text{new}} \equiv x$ **or not** `validate(`$x_{\text{near}}, x_{\text{new}}$`)`;
- **return** `validate(`$x_{\text{near}}, x_{\text{new}}$`)`;

Function `nearest(`T, x, c`)`
- $x_{\text{near}} \leftarrow$
- $\arg\min_{v \in V_T} \left\{ w_x ||x - x_v||^2 + w_c |c - c_v|^2 \mid c_v + \hat{c}(x_v, x) < c \right\}$;
- **return** x_{near};

Global optimization-based planning


goal
start
2
1
0
−1
−2
−4
−3
−2
−1
0
1
2
3
4

Motion planning as a (nonconvex) optimization



fixed number of samples

$$\min_{q_0,\ldots,q_N} \sum_{n=0}^{N-1} |q_{n+1} - q_n|_2^2$$

$$\text{subject to} \quad q_0 = q_{start}$$

$$q_N = q_{goal}$$

collision-avoidance → $|q_n|_1 \geq 1 \quad \forall n$

(outside the L^1 ball)

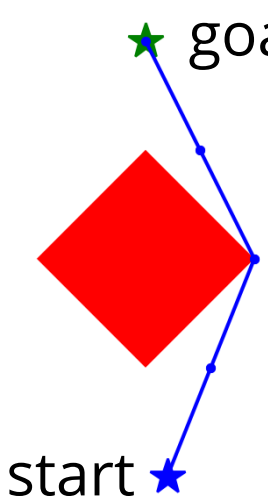
nonconvex

Motion planning with Graphs of Convex Sets (GCS)

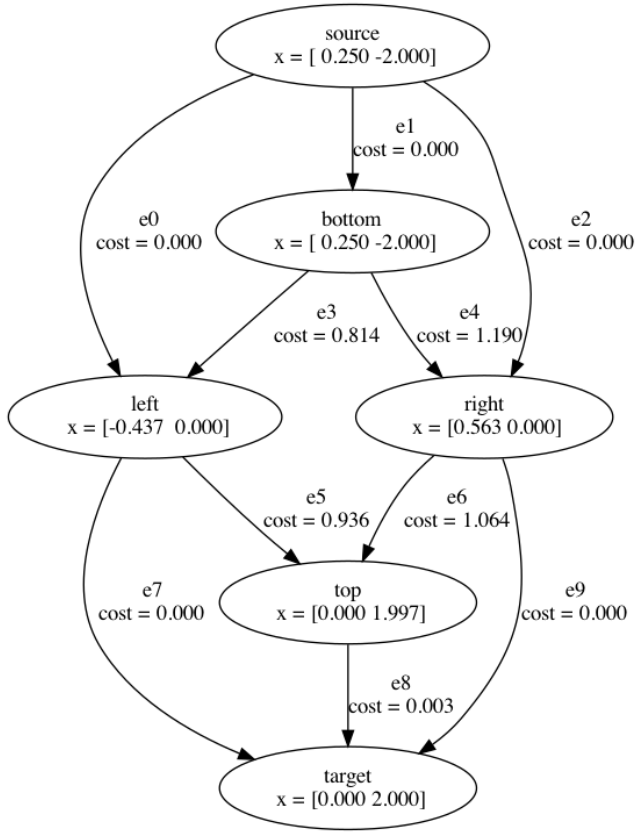
Motion Planning around Obstacles with Convex Optimization.
Tobia Marcucci, Mark Petersen, David von Wrangel, Russ Tedrake.
Available at: https://arxiv.org/abs/2205.04422
Accepted for publication in *Science Robotics*

Claims:

- Find *better* plans *faster* than sampling-based planners
- Avoid local minima from trajectory optimization
- Can guarantee paths are collision-free
- Naturally handles dynamic limits/constraints
- Scales to big problems (e.g. multiple arms)



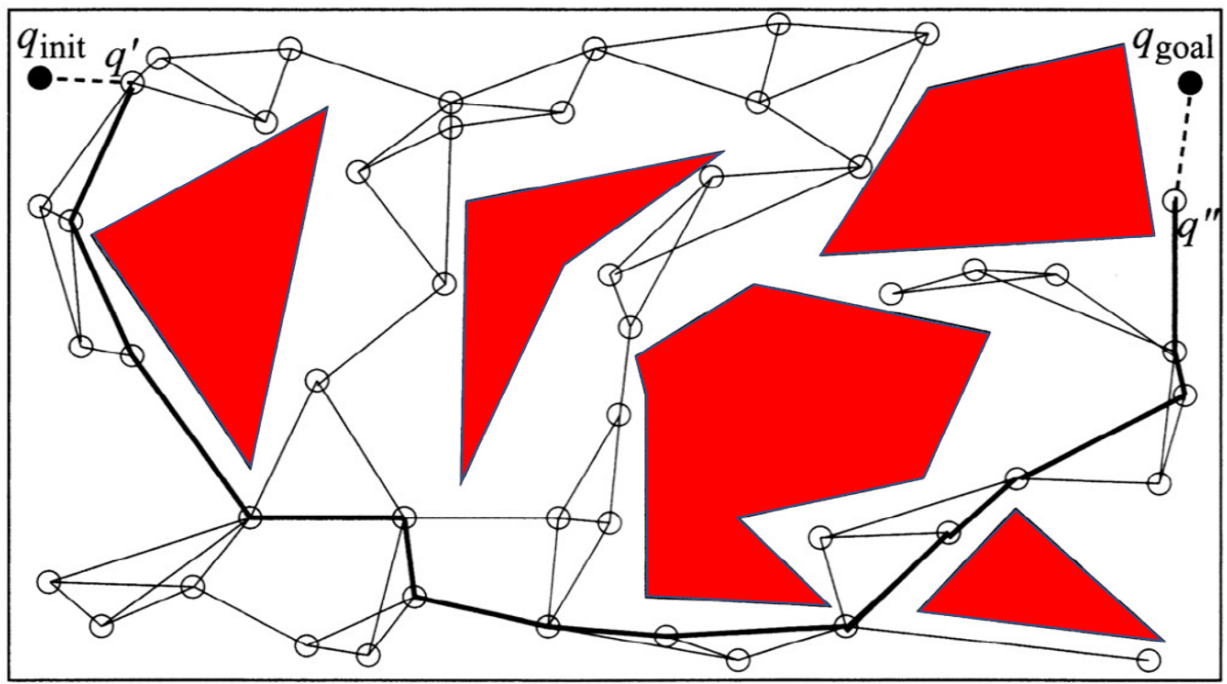
$$\ell_{i,j}(x_i, x_j) = |x_i - x_j|_2$$



https://people.csail.mit.edu/russt/uploads/bimanual_cluttered_crossover.html

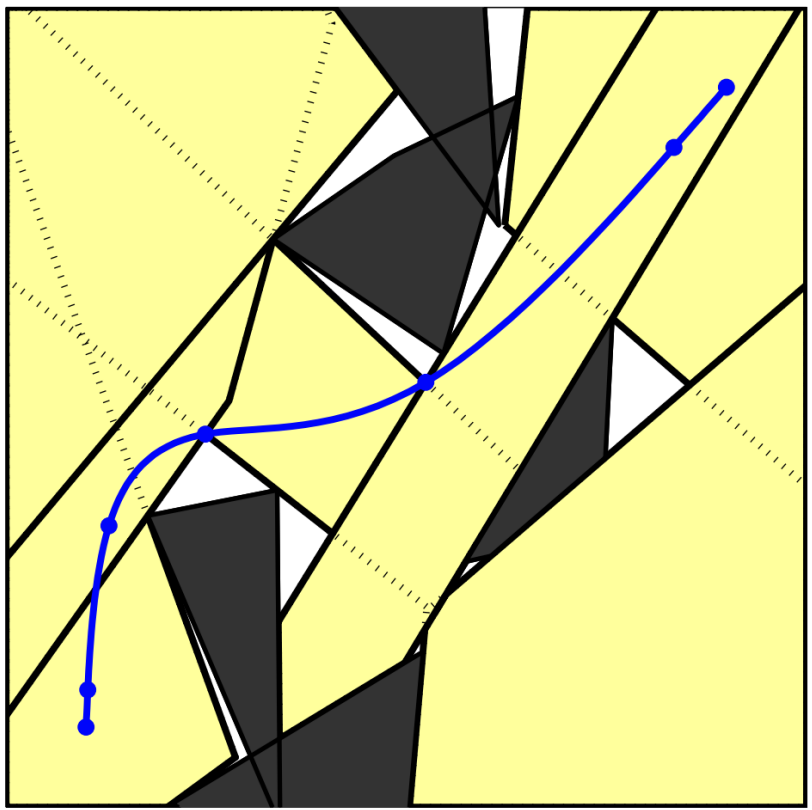
GCS Trajectory Optimization

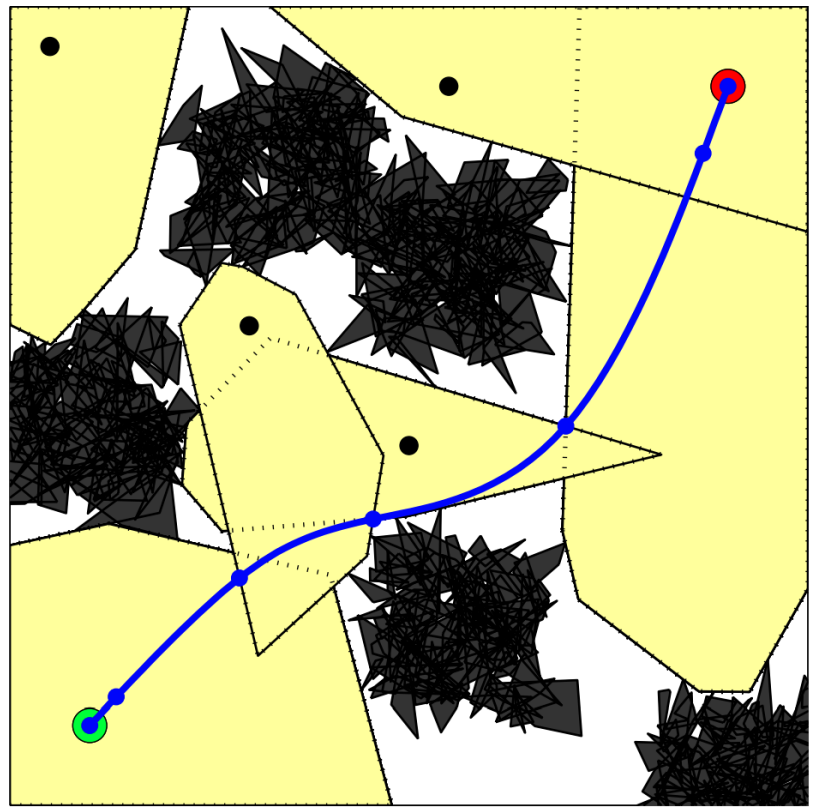
Sampling-based motion planning



The Probabilistic Roadmap (PRM)
from Choset, Howie M., et al. *Principles of robot motion: theory, algorithms, and implementation*. MIT press, 2005

- PRM: A roadmap of points (vertices) connected by line segments
- GCS: A roadmap of *convex sets* (vertices) of continuous curves connected by continuity constraints

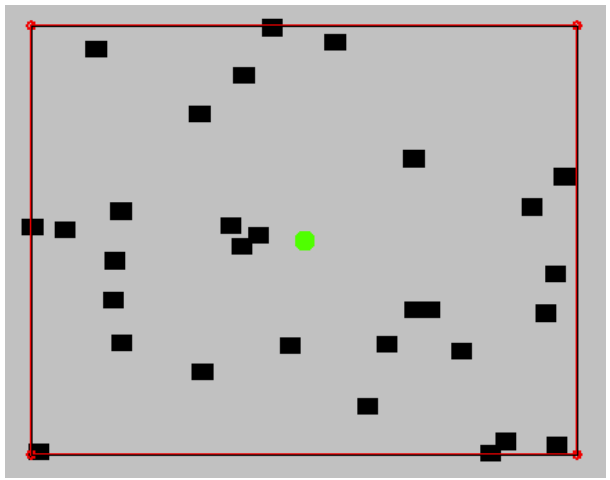


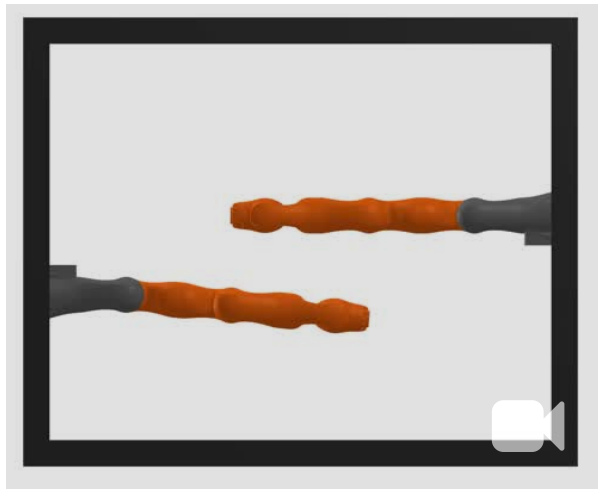


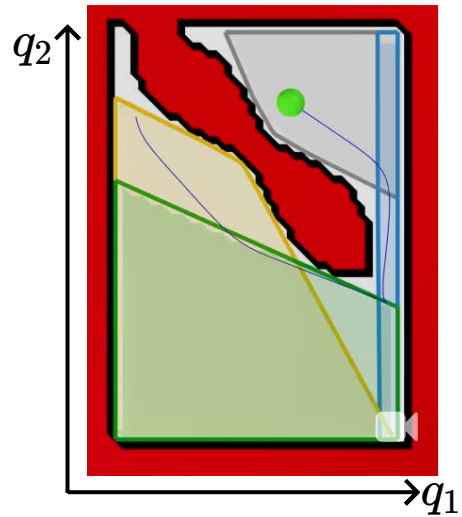
Key ingredients

- (Approximate) convex decomposition of collision-free configuration space. (aka "*IRIS*")
- Efficient algorithms for solving shortest paths on "Graphs of Convex Sets" (aka "*GCS*")
- Convex optimization over continuous curves w/ time scaling ($\Rightarrow$ "*GCS Trajectory Optimization*")

IRIS for Configuration-space regions

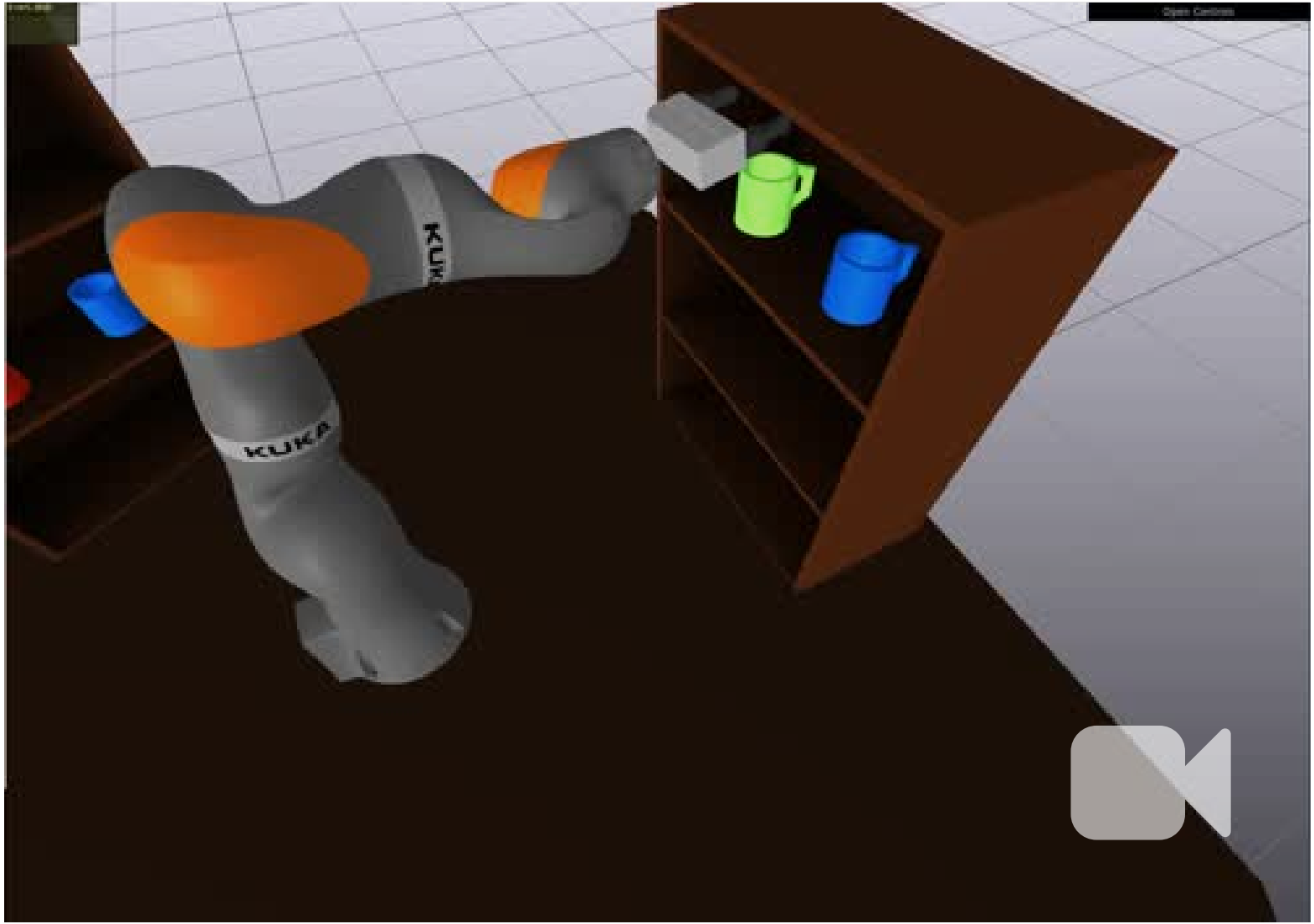




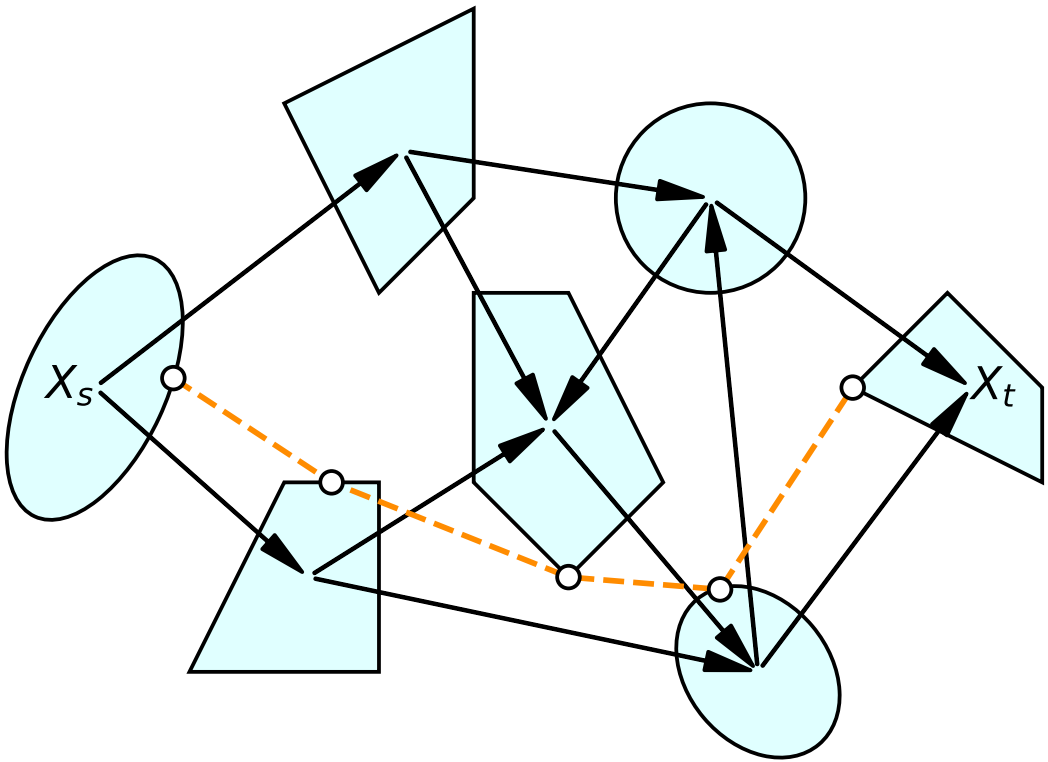


- IRIS (Fast approximate convex segmentation). *Deits and Tedrake, 2014*
- New versions work in configuration space:
  - Nonlinear optimization for speed. *IRIS-NP*
  - Sums-of-squares for rigorous certification. *C-IRIS*

https://www.desmos.com/calculator/kol7mvpgke

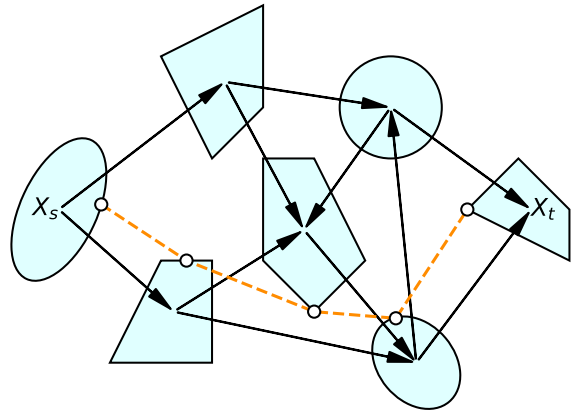

KUKA
KUKA

Graphs of Convex Sets



Note: The blue regions are *not* obstacles.

Graphs of Convex Sets

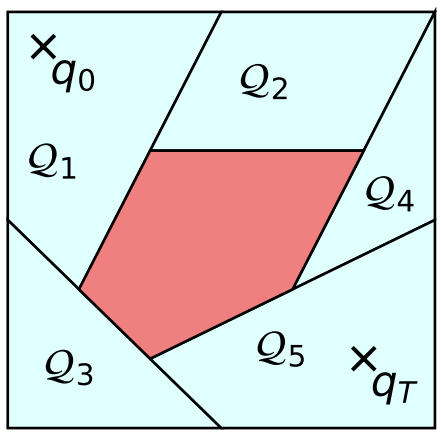


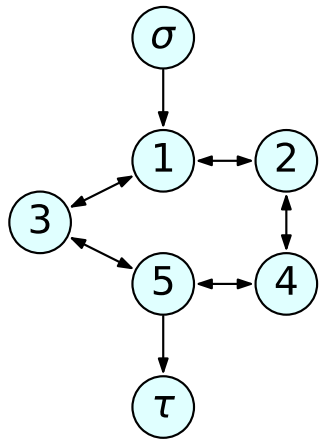
Mixed-integer formulation with a very tight **convex** relaxation

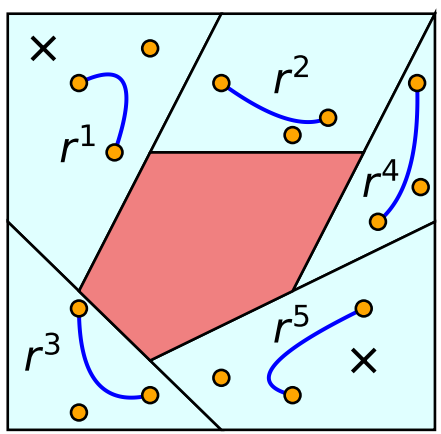
- Efficient branch and bound, or
- In practice we *only solve the convex relaxation* and round

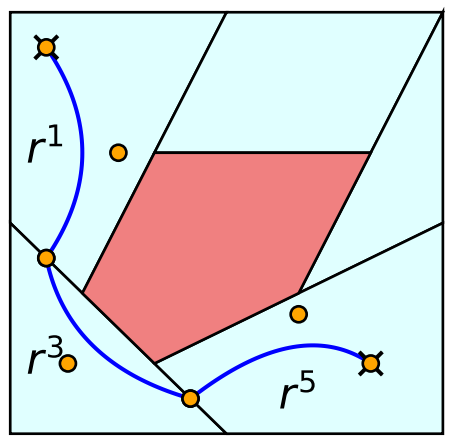
Main idea: Multiply constraints + Perspective function machinery

GCS Trajectory Optimization

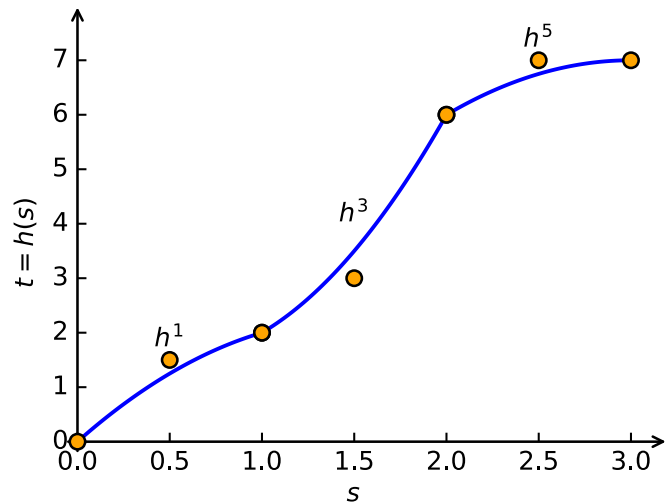


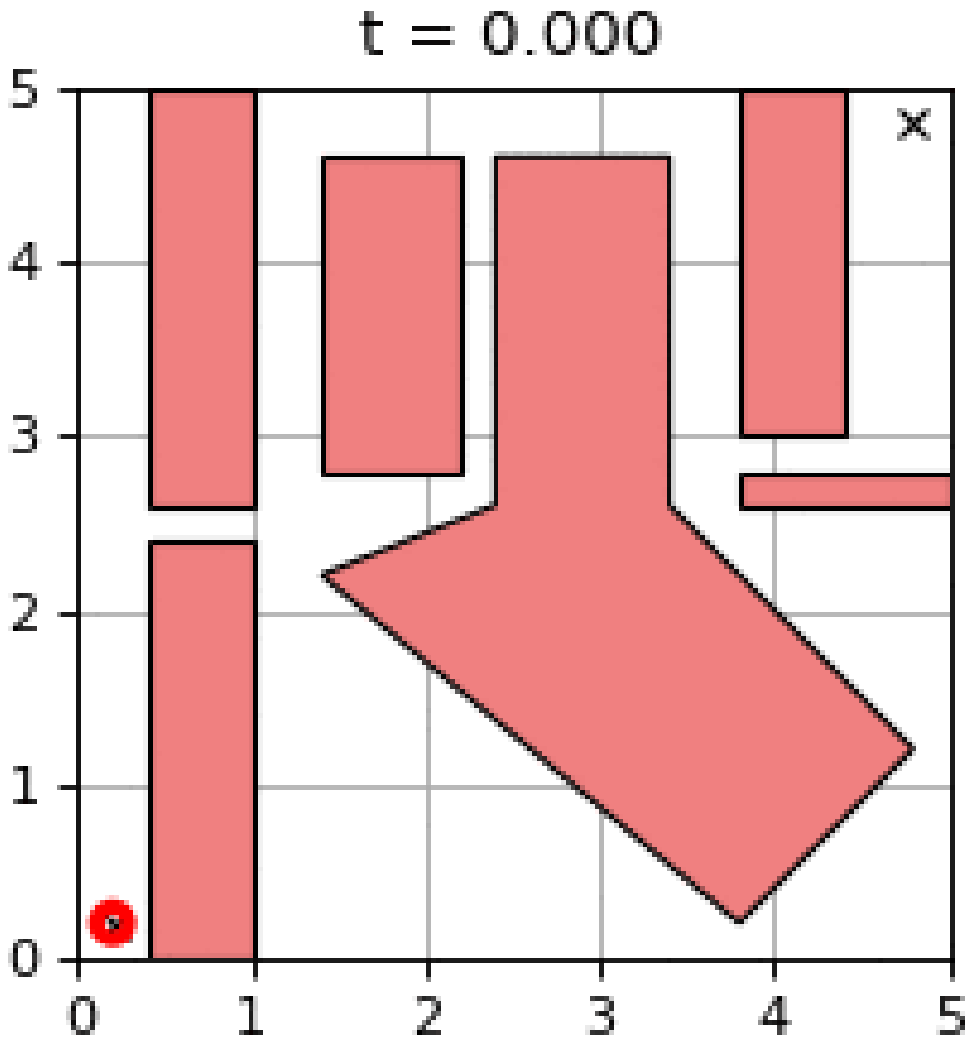


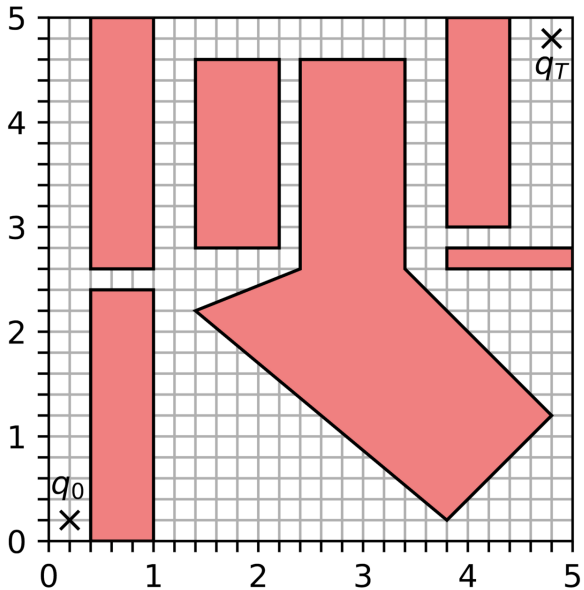


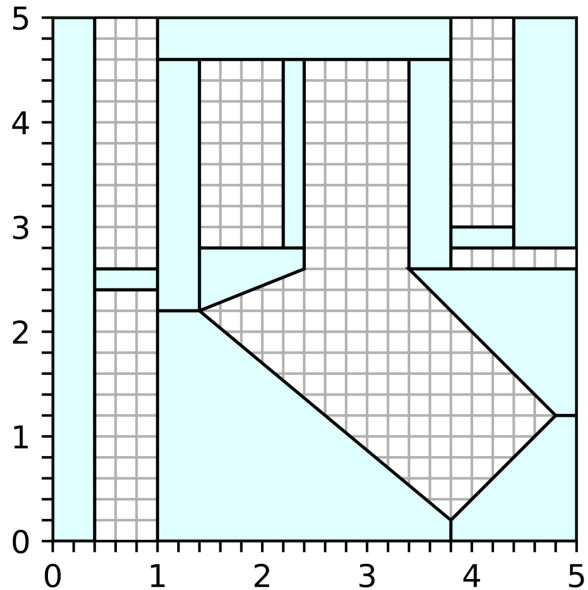


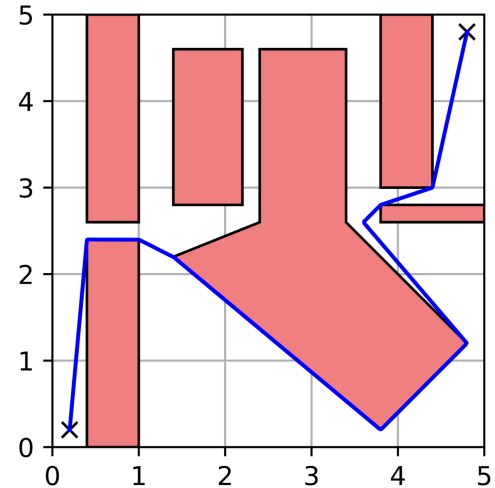
+ time-rescaling

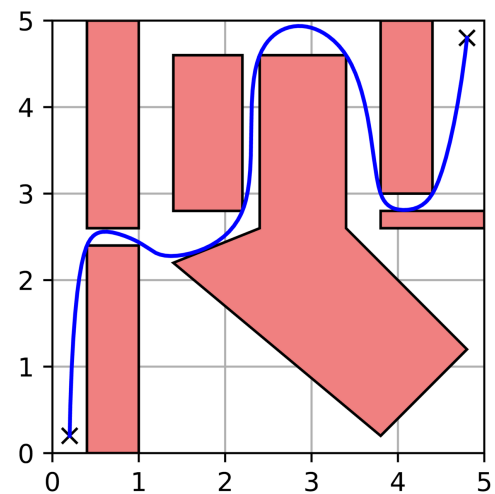



t = 0.000
x
0
1
2
3
4
5


q_T
q_0



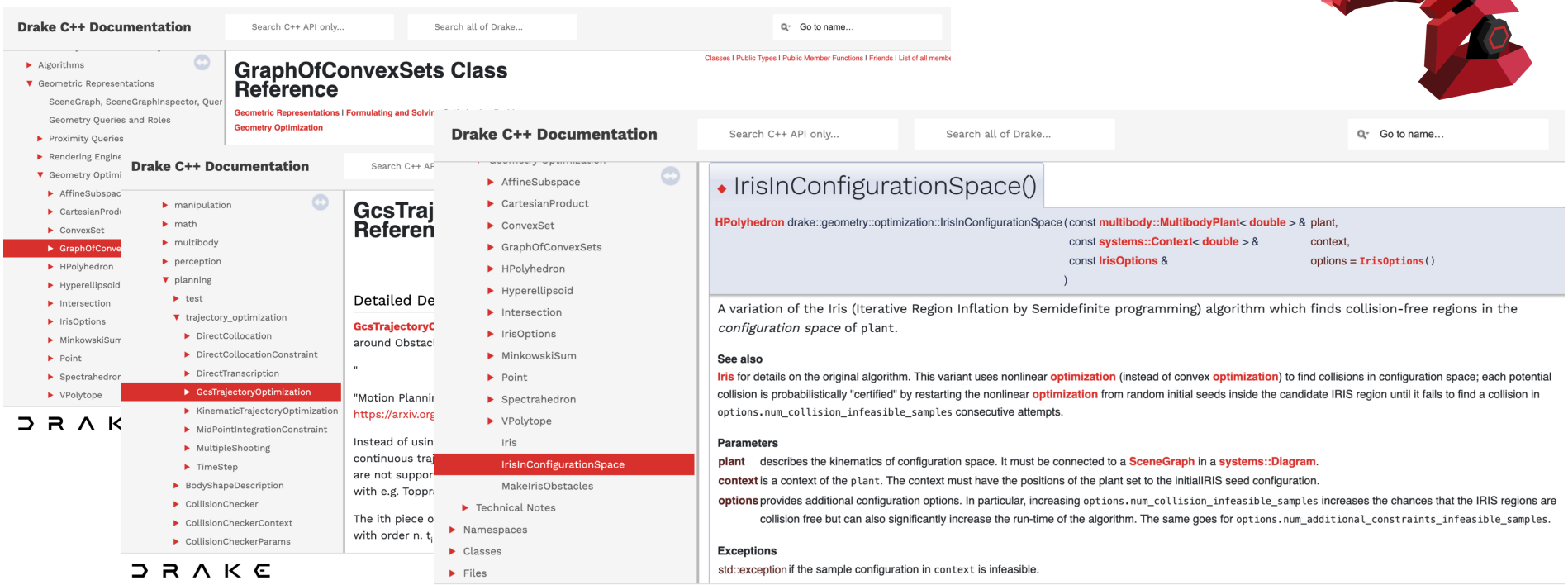




GCS Trajectory Optimization

Transitioning from basic research to real use cases

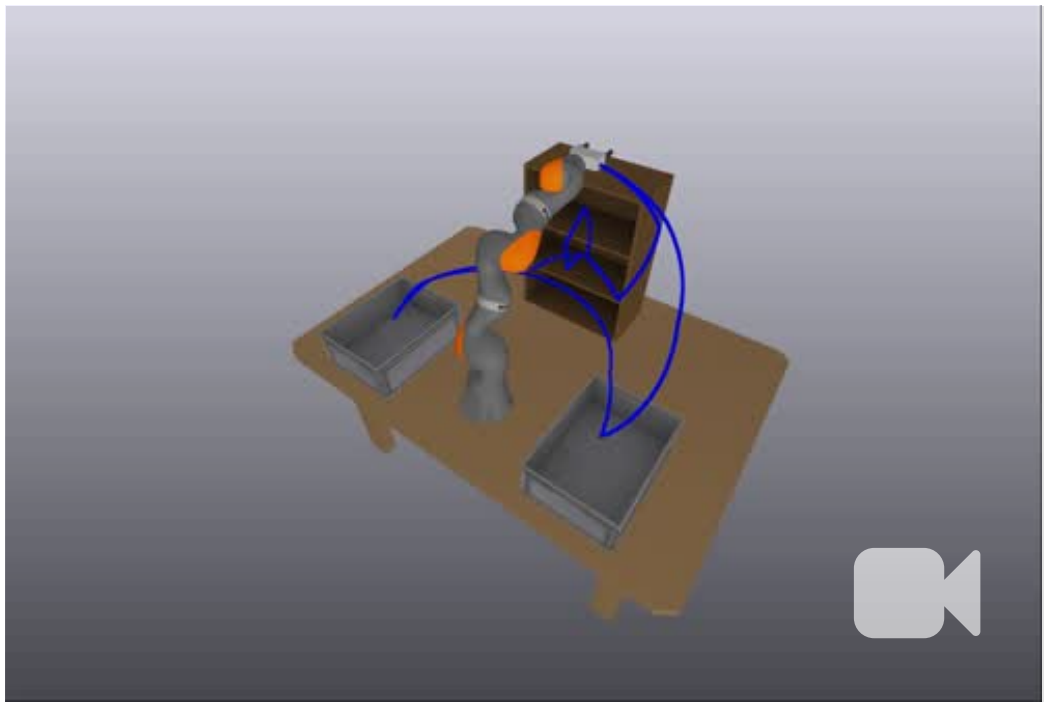




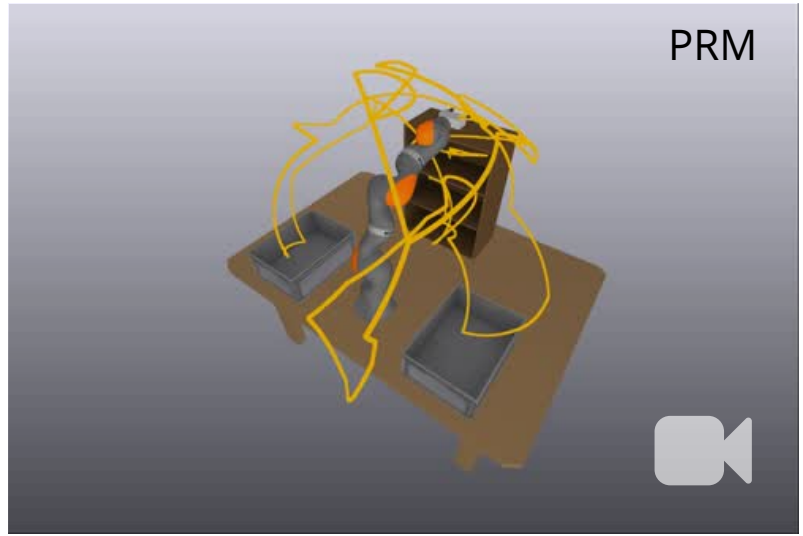
duration

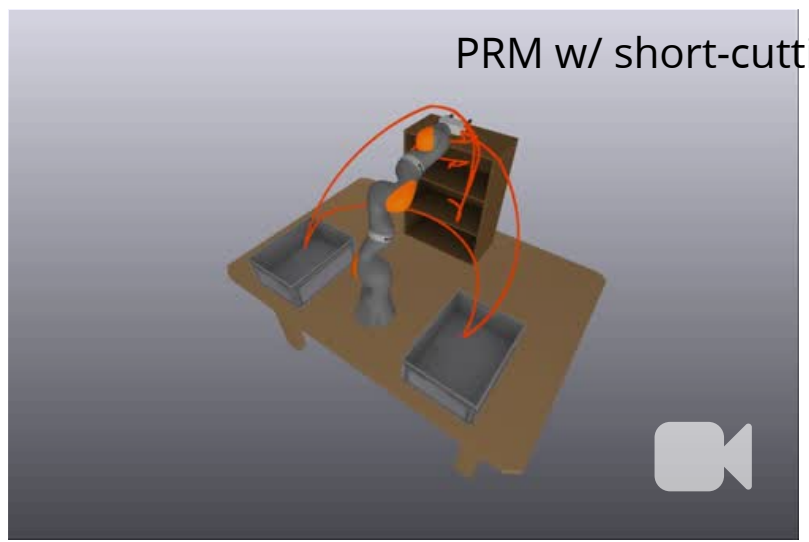
$$\min \quad aT$$

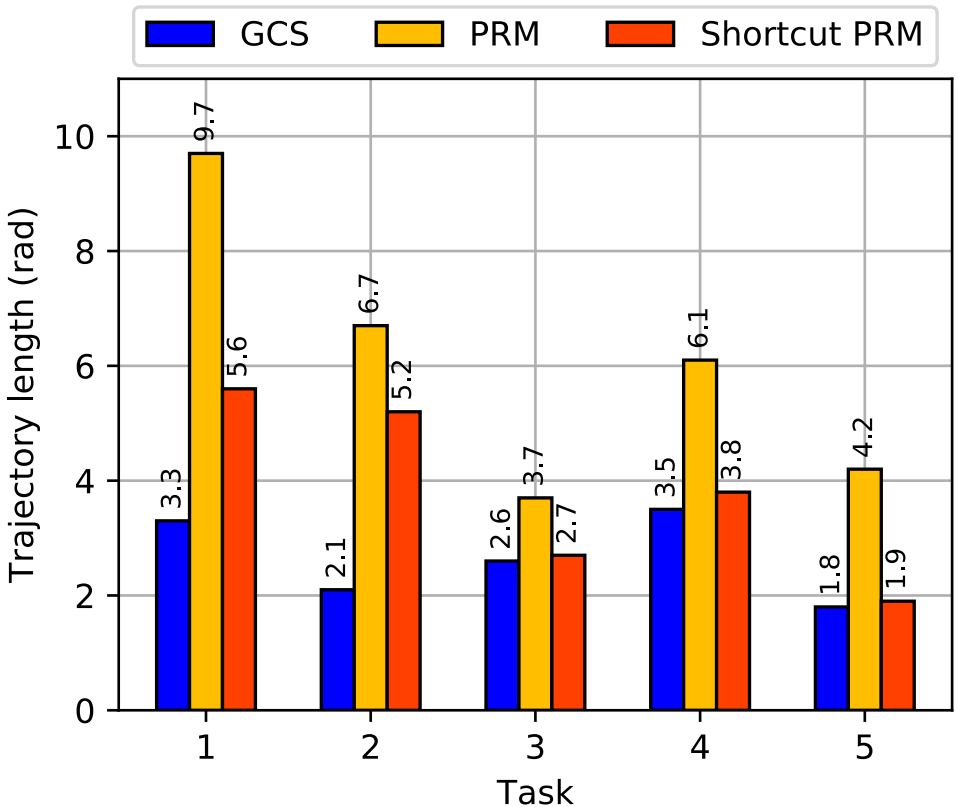
s

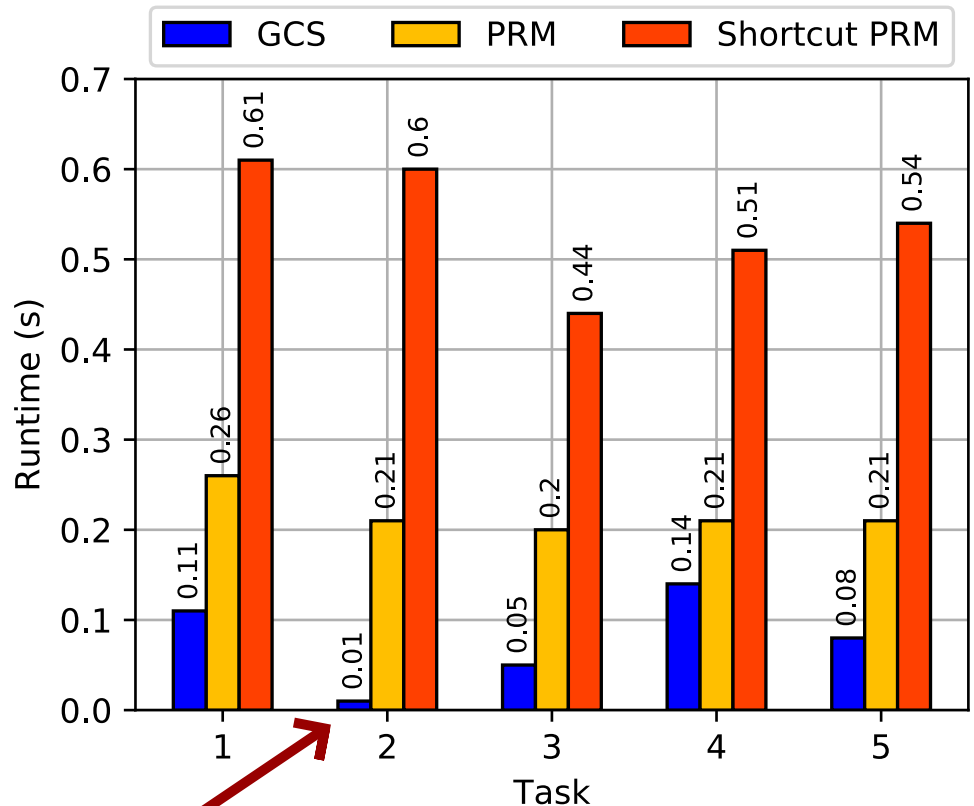


Graph of Convex Sets (GCS)

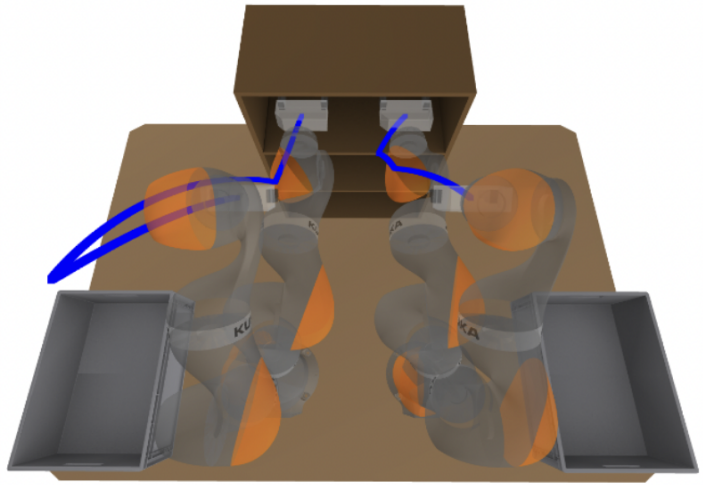




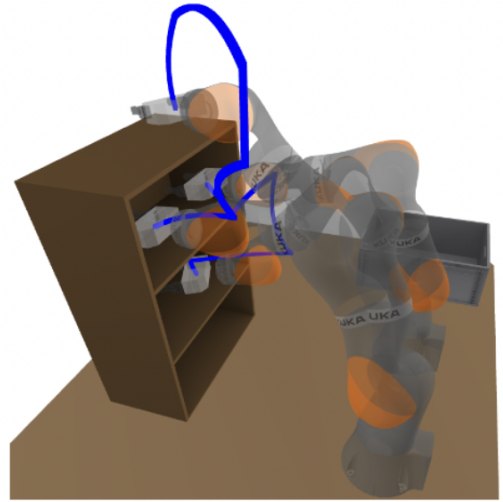




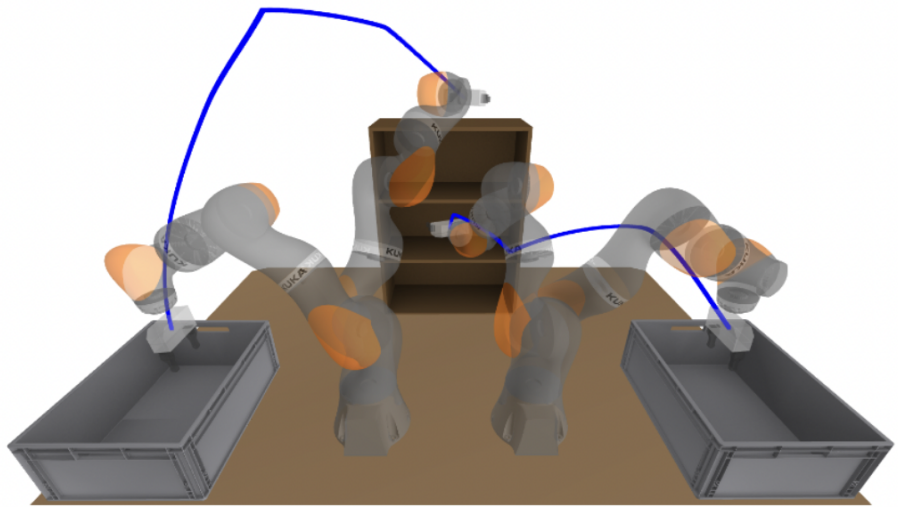
Preprocessor now makes easy optimizations fast!



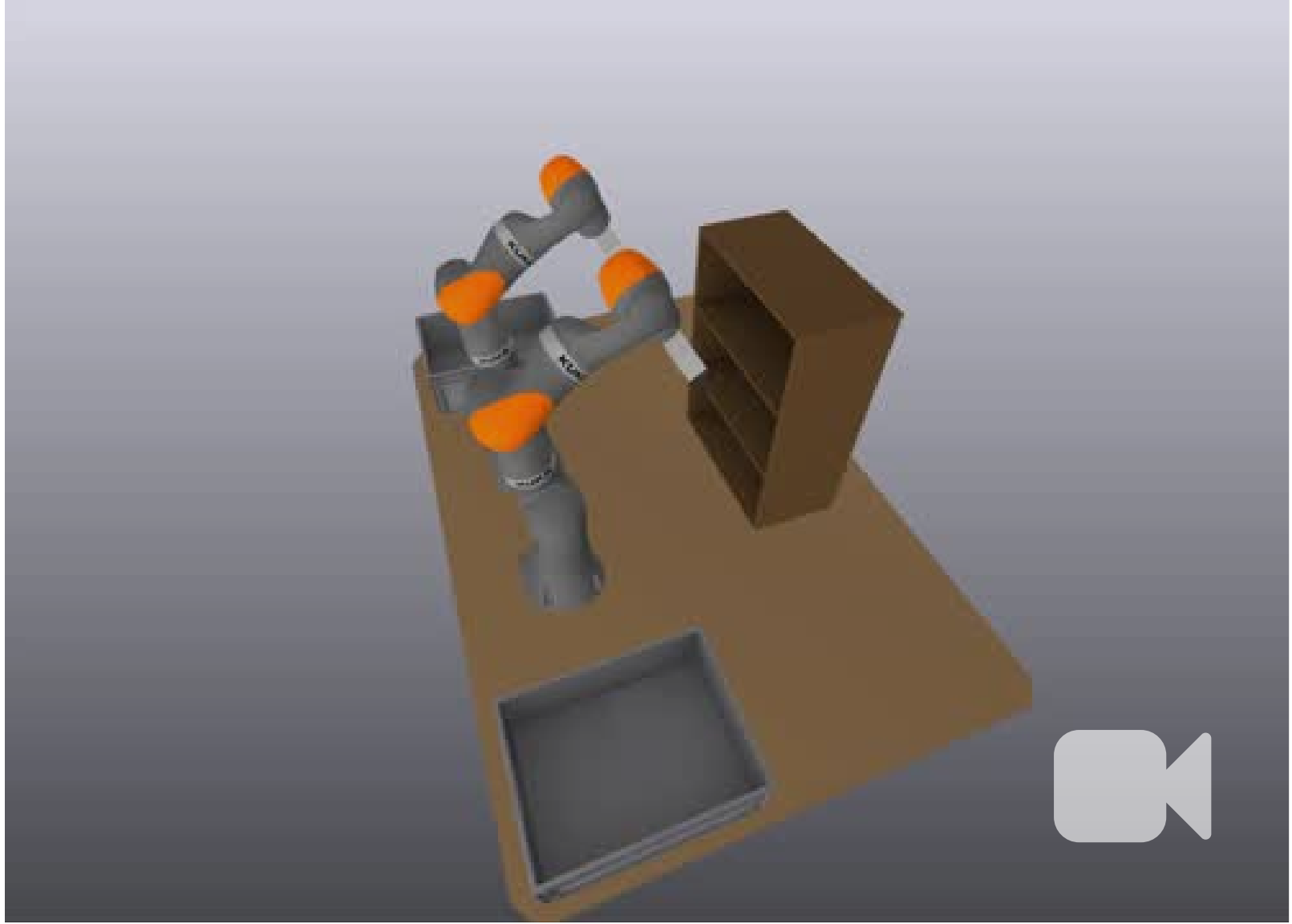
(a)



(b)



(c)





https://people.csail.mit.edu/russt/uploads/gcs_uav_smoothed_trajectory_tracking.html

Default playback at .25x

An early success story

